



算法设计与分析

项目实验报告

选题：题目 6——股票交易最优策略（动态规划）

小组成员： 杨博文、张瑾然、宋雨璇

专 业： 计算机科学与技术

年 级： 2023 级

人工智能学院

目录

一、问题背景与定义	1
1.1 问题背景	1
1.2 问题特性分析	1
1.2.1 交易行为特性	1
1.2.2 状态空间分析	2
1.3 算法选择与优化	2
二、算法核心设计步骤（动态规划）	2
2.1 动态规划框架构建	2
2.2 两种实现方案	3
三、算法实现与模块设计（辅助实现）	5
3.1 数据生成模块	5
3.2 路径回溯设计	6
3.3 性能测试方案	7
四、两种动态规划方案的对比分析与评估	9
4.1 方案设计思路对比	9
4.2 实验结果对比	9
4.3 方案的优缺点分析	10
4.4 方案总结	10
五、测试与分析	11
5.1 时间复杂度分析	11
5.2 空间复杂度分析	11
5.3 测试环境与数据	12
5.4 性能测试结果	12
六、总结与展望	13
6.1 实验总结	13
6.2 算法优缺点分析	13
6.3 未来改进的方向	14
6.4 量化交易的延伸思考	14
6.5 课程学习体会	14
七、参考文献	15

一、问题背景与定义

1.1 问题背景:

股票交易优化问题是一个经典的金融决策问题,在算法设计与分析领域具有重要的研究价值。选题6设定了一个特定的交易环境:给定一个整数数组 `prices`, 其中 `prices[i]` 表示第 i 天的股票价格 (单位股票); 整数 `fee` 代表交易股票的手续费用。在这个交易系统中, 存在以下几个关键约束条件:

- (1) **交易限制:** 每天最多可以完成 10 笔交易 (包括买与卖), 且每笔交易只能操作 1 股
- (2) **手续费机制:** 每笔交易都需要支付固定手续费 `fee`
- (3) **持仓限制:** 最多只能持有 20 个单位股票, 超过后将无法继续购买
- (4) **初始条件:** 初始持有股票单位为 0
- (5) **数据规模:** 随机生成的 `prices` 数组, 元素个数 $1000 \leq N \leq 2000$, 价格范围 $1 \leq prices[i] \leq 100$, 手续费 $10 \leq fee \leq 20$

目标函数是最大化最终利润, 计算方式为:

最后一天拥有的现金 + 最后一天持有的股票数量 $\times$ 当天股票价格

1.2. 问题特性分析

1.2.1 交易行为特性

在分析这个问题时, 我们首先需要理解交易行为的几个重要特性:

特性一: 同一天内不会同时进行买卖操作

- 数学证明:

假设在某一天, 我们同时进行买入 m 股和卖出 n 股的操作 ($m, n > 0$)。由于同一天的股票价格相同, 设当天价格为 p , 手续费为 fee 。

- 净买入量: $m - n$ 股
- 总交易成本: $(m + n) \times fee$
- 净现金变化: $-(m - n) \times p - (m + n) \times fee$
- ✓ 如果 $m > n$ (净买入), 这等价于直接买入 $(m - n)$ 股, 但多支付了 $2n \times fee$ 的手续费 (因为卖出 n 股又买入 n 股是冗余操作)。
- ✓ 如果 $n > m$ (净卖出), 这等价于直接卖出 $(n - m)$ 股, 但多支付了 $2m \times fee$ 的手续费。
- ✓ 如果 $m = n$, 则没有任何净交易, 却支付了 $2m \times fee$ 的手续费。

结论: 因此, **同一天内同时买卖股票总是不如仅进行净买卖操作更优**。这一特性使得我们可以简化交易决策, 每天只需考虑净买入或净卖出, 而无需考虑同时买卖的情况。

特性二: 每日最优交易量要么为 0, 要么为最大值 10 股

- 直观理解:

股票交易的核心原则是“低买高卖”。如果我们决定在某天买入股票, 说明我们认为未来价格会上涨, 那么买入越多获利潜力越大 (在持仓限制内)。同理, 如果决定卖出, 说明我们认为当前价格较高, 卖出越多锁定利润越多。

- 数学证明:

考虑某一天 i , 假设我们决定进行买入操作。设当天价格为 p_i , 手续费为 fee 。

- 买入 k 股的收益函数 (相对于不操作) 可以表示为:

$$\Delta = \sum_{j=1}^k [(p_{i+t_j} - p_i) - 2 * fee] \quad (\text{其中 } t_j \text{ 是各股卖出的时间})$$

由于手续费是固定成本, 在决定买入时, 我们预期未来价格足够高以覆盖手续费。因此,

在持仓限制内，买入越多潜在收益越大。同理适用于卖出操作。

结论：每天最多交易 10 股（买入或卖出），且每笔交易 1 股，最优策略通常会在允许范围内最大化交易量（10 股）或不交易（0 股）。中间值（如交易 5 股）通常不如全量交易有利，除非有特殊的持仓限制考虑。**每天持股状态为 0，10，20。**

1.2.2 状态空间分析

这个问题具有明显的**马尔可夫性质**——第 i 天的最大利润只依赖于第 $i-1$ 天的状态和当天的决策。这使得动态规划成为解决此问题的理想方法。

状态定义：我们可以定义 $dp[i][s]$ 表示第 i 天结束时持有 s 股股票时的**最大现金余额**。由于持仓限制为 20 股， s 的取值范围是 0 到 20，共 21 种可能状态。**然而，根据分析，其实只有三种状态——0，10，20。**

状态转移：对于每个状态 $dp[i][s]$ ，它可以由前一天的多种可能状态转移而来：

1. 不操作： $dp[i-1][s]$
2. 买入 k 股： $dp[i-1][s-k] - k \times \text{prices}[i] - k \times \text{fee}$ ($1 \leq k \leq \min(10, s)$)
3. 卖出 k 股： $dp[i-1][s+k] + k \times \text{prices}[i] - k \times \text{fee}$ ($1 \leq k \leq \min(10, 20-s)$)

边界条件：

$dp[0][0] = 0$ （初始现金为 0，无持仓）；

$dp[0][s] = -10000$ for $s \neq 0$ （初始不可能持有股票）

目标函数：

最终的最大利润为 $\max_{0 \leq s \leq 20} [dp[N][s] + s \times \text{prices}[N]]$

1.3 算法选择与优化

首先，我们来谈论蛮力法的不可行性。理论上，我们的确可以简单地考虑所有可能的交易序列并且通过比较得出最优解。但由于以下原因，蛮力法并不适用：**第一，蛮力法的时间效率极低：**尽管经过分析，每天最少只需要考虑 3 种持仓可能（0，10，20 股），那么 N 天需要枚举的个数为 3^N ，对于 $N=2000$ 而言，这是一个天文数字。蛮力法经过枚举之后需要将每种情况的结果进行记录，并且比较，比较同样需要很多时间。**第二，蛮力法的空间效率极低：**蛮力法经过枚举之后需要将每种情况的结果进行记录，并且比较，在得到最优解之前同时也必须将每种序列记录下来，这会导致空间内存开销过大。综上考虑，蛮力法在实践中可行性较低，故我们应该采取更优的算法。

其次，我们来考虑动态规划的可行性与优势。动态规划方法有效解决了蛮力法的不足，因为该问题具备动态规划解决问题的性质：**首先，问题满足最优子结构：**问题的最优解包含子问题的最优解。第 i 天的最优状态只依赖于第 $i-1$ 天的最优状态。**其次，子问题具有重叠性：**相同的子问题会被多次计算，动态规划通过表格存储避免了重复计算。**最后，问题求解过程具备多项式时间复杂度：**我们按照 3 状态方案来考虑，状态数为 $O(N \times S)$ ，其中 $S=3$ （持仓状态数），决策数为 $O(K)$ ， $K=10$ （最大交易量）。总复杂度为 $O(N \times S \times K)$ ，对于 $N=2000$ ，约为 60,000 次计算，是可行的。

目前，通过对这个股票交易问题的深入分析，我们揭示了其内在结构和特性，证明了关键性质（同一天不同时买卖、最优交易量为 0 或最大值 10），并建立了基于动态规划的高效解决方案。这种分析不仅解决了具体问题，也为理解更复杂的金融决策问题提供了方法论基础。

二、算法核心设计步骤（动态规划）

2.1 动态规划框架构建

算法采用动态规划方法解决股票交易优化问题，核心设计如下：

状态定义：定义二维数组 $dp[i][s]$ ，其中 i 表示天数 ($1 \leq i \leq N$)， s 表示当前持有股票数量 ($0 \leq s \leq 20$)。 $dp[i][s]$ 的值表示第 i 天持有 s 股时的最大现金余额。

状态转移方程：对于每个状态，考虑三种操作可能：

1. 不操作：

$$dp[i][s] = dp[i-1][s]$$

2. 买入 k 股 ($1 \leq k \leq 10$)：

$$dp[i][s] = \max(dp[i][s], dp[i-1][s-k] - k \times \text{prices}[i] - k \times \text{fee})$$

3. 卖出 k 股 ($1 \leq k \leq 10$)：

$$dp[i][s] = \max(dp[i][s], dp[i-1][s+k] + k \times \text{prices}[i] - k \times \text{fee})$$

初始化与结果计算：

初始状态： $dp[0][0] = 0$ ，其他 $dp[0][s] = -10000$

最终结果： $\max(dp[N][s] + s \times \text{prices}[N]), 0 \leq s \leq 20$

2.2 两种实现方案

根据问题规模设计了两种实现方案：

■ 小规模方案（3 状态）

对于该方案，首先建立动态规划记录表用以记录子问题最优解以及交易路径。其次，开始进行自下而上的动态递推：分别考虑第 i 天持有 0 股，10 股，20 股的状态是如何由前一天转移得到的。

例如：某一天是 0 股只可能是前一天持有 0 股并且在当天采取不买不卖的交易策略，亦或是前一天持有 10 股并且于当天采取了卖出 10 股的策略；在这种情况下需要将这两种情况比较并且让资产最大化，将这一天应该得到 0 股采取的最优策略记录在交易路径表中。

又例如：某一天是 10 股可能是前一天持有 0 股当天采取购买 10 股的交易策略，亦或是前一天持有 10 股当天采取不卖不卖的交易策略，亦或是前一天持有 20 股当天采取卖出 10 股的策略。需要比较三种情况哪种情况下资产价值最高，并且将使得这天持有 10 股的最优策略记录在交易路径表中。

//建立动态规划记录表

```
vector<vector<int>> dp(N + 1, vector<int>(3, -10000));  
vector<vector<pair<int, string>>> path(N + 1, vector<pair<int, string>>(3));  
dp[0][0] = 0;
```

//动态规划迭代过程以及交易路径记录

```
for (int i = 1; i <= N; i++)  
{  
    dp[i][0] = max({ dp[i - 1][0], dp[i - 1][1] + prices[i] * 10 - fee * 10 });  
    if (dp[i - 1][0] >= (dp[i - 1][1] + prices[i] * 10 - fee * 10))  
    {  
        path[i][0] = { 0, "HOLD" };  
    }  
    else {  
        path[i][0] = { 1, "SELL 10" };  
    }  
}
```

```

    }
    dp[i][1] = max({ dp[i - 1][1], dp[i - 1][0] - prices[i] * 10 - fee * 10, dp[i - 1][2] + prices[i] * 10 - fee * 10 });

    if (dp[i - 1][1] >= (dp[i - 1][0] - prices[i] * 10 - fee * 10) && dp[i - 1][1] >= (dp[i - 1][2] + prices[i] * 10 - fee * 10))
    {
        path[i][1] = { 1, "HOLD" };
    }
    else if ((dp[i - 1][0] - prices[i] * 10 - fee * 10) > dp[i - 1][1] && (dp[i - 1][0] - prices[i] * 10 - fee * 10) > (dp[i - 1][2] + prices[i] * 10 - fee * 10))
    {
        path[i][1] = { 0, "BUY 10" };
    }
    else
    {
        path[i][1] = { 2, "SELL 10" };
    }
    dp[i][2] = max({ dp[i - 1][2], dp[i - 1][1] - prices[i] * 10 - fee * 10 });
    if (dp[i - 1][2] >= (dp[i - 1][1] - prices[i] * 10 - fee * 10))
    {
        path[i][2] = { 2, "HOLD" };
    }
    else
    {
        path[i][2] = { 1, "BUY 10" };
    }
}}

```

■ 大规模方案（21 状态）

我们使用大规模的方案并最后进行路径回溯来验证我们一开始所说的特性二（每日最优交易量要么为 0，要么为最大值 10 股）的结论。动态规划每天操作 0-10 股，看看这种更精细的方案效果情况如何。

此时，遇到了两个问题：首先是边界溢出的问题，如果单纯的考虑每天操作 k 股，那么有可能产生不合实际的迭代计算（例如：某一天持有 0 股由前一天持有 -k 股并且于当天购入 k 股得到）。其次是递推公式的问题，需要我们遍历的情况增加，对于同一天持有 k 股，有多种转移情况，不能像 3 状态方案那样使用 if-else 分支遍历，计算效率低而且编写代码也不方便。所以，采用双重循环，对于每一种持仓状态需要一边进行遍历交易次数，一边设置最大值进行比较和更新。这样既方便代码的编写，也减少了内存的占用，每天的一种持仓状态的最优策略只需要记录一次（在迭代中更新）。

//建立动态规划记录表

```

vector<vector<int>> dp(N + 1, vector<int>(21, -10000));
vector<vector<pair<int, string>>> path(N + 1, vector<pair<int, string>>(21));
dp[0][0] = 0;

```

```

//动态规划迭代过程以及交易路径记录
for (int i = 1; i <= N; i++)
{
//当天不买不卖
    for (int stock = 0; stock <= 20; stock++)
    {
        int stay = dp[i - 1][stock];
        dp[i][stock] = stay;
        path[i][stock] = { stock, "HOLD" };
//当天购买
        for (int k = 1; k <= 10; k++)
        {
            int prev = stock - k;
            if (prev >= 0 && dp[i - 1][prev] != -10000)
            {
                int cost = prices[i - 1] * k + fee * k;
                int val = dp[i - 1][prev] - cost;
                if (val > dp[i][stock])
                {
                    dp[i][stock] = val;
                    path[i][stock] = { prev, "BUY " + to_string(k) };
                }
            }
        }
//当天卖出
        for (int k = 1; k <= 10; k++)
        {
            int prev = stock + k;
            if (prev <= 20 && dp[i - 1][prev] != -10000)
            {
                int income = prices[i - 1] * k - fee * k;
                int val = dp[i - 1][prev] + income;
                if (val > dp[i][stock])
                {
                    dp[i][stock] = val;
                    path[i][stock] = { prev, "SELL " + to_string(k) };
                }
            }
        }
    }
}

```

三、算法实现与模块设计（辅助实现）

3.1 数据生成模块

为了更全面地刻画股市波动地真实性，我们采取了两种价格随机生成策略。

方案 1，我们采取了正态分布价格生成策略。正态分布的价格生成模拟了**有效市场假说**下的价格行为，反映了 3 点市场特征：首先是均值回归特性，价格总是在 $\mu=50$ 附近波动，反映了大众对于这支股票的日常基本估值和预期。其次是极端事件的小概率发生可能性，股市很有可能由于重大自然灾害，科技突破，政治动荡等不确定性因素而产生类似“黑色星期五”这样子的金融市场剧烈波动。再者，固定标准差 $\sigma=20$ 模拟了波动率相对稳定的市场环境，股价总是波动的，但是在一个较小的时系内部大概率是平稳波动的。

方案 2，我们采取了均匀分布价格生成策略。均匀分布的价格生成模拟了**极端市场环境**，体现了以下 3 点市场特征：首先是无记忆性，认为股价是完全随机游走的，这符合现实世界种股价无法预测的特性。其次是肥尾现象，因为现实世界中极端价格出现的概率尽管小，但是远远高于正态分布下的概率。再者，这种情况更加吻合金融危机期间、小盘不稳定股票的价格行为。

该模块的实现细节如下：

```
//<方案一>正态随机价格
normal_distribution<> price_dist(50.0, 20.0);
for (int i = 0; i <= N; i++)
{
    double val = price_dist(gen);
    prices[i] = clamp_price(val);
}

//<方案二>随机随机价格
uniform_int_distribution<> distrib(1,100);
for(int i=0;i<=N;i++)
{
    prices[i]=distrib(gen);
}
```

3.2 路径回溯设计（以 3 状态方案为例）

动态规划表数据结构：

```
vector<vector<pair<int, string>>> path(N+1, vector<pair<int, string>>(21));
```

其中 **path[day][stock]** 记录到达状态 **dp[i][stock]** 的最优前驱状态和操作描述，包含两个元素：**prev_stock** 表示前一天的持仓量；**action** 表示当天的操作描述（如“BUY”、“SELL”、“HOLD”）

关于**路径回溯过程**，首先，在最后一天的三种持有状态中选择资产价值最大者；然后，根据 **path** 数组逆向追踪每日操作；最后，反转操作序列得到正向交易记录。

```
//选择第 N 天的最大资产价值
res = max({ dp[N][0], dp[N][1] + 10 * prices[N], dp[N][2] + 20 * prices[N] });
//回溯交易路径
cout << "最终最大利润：" << res << endl;

cout << "回溯路径：" << endl;
```



```

vector<string> actions;
int stock = best_stock;
for (int day = N; day >= 1; day--)
{
    pair<int, string>pa = path[day][stock];
    int prev_stock = pa.first;
    string action = pa.second;
    string record = "Day " + to_string(day) + ": " + action + " (Stock: " +
to_string(stock * 10) + ") Prices=" + to_string(prices[day - 1]);
    actions.push_back(record);
    stock = prev_stock;
}
//反转操作序列
reverse(actions.begin(), actions.end());
for (size_t i = 0; i < actions.size(); i++)
{
    cout << actions[i] << endl;
}
return 0;
}

```

输出格式如下：

```

N:1490
fee:20
最终最大利润：43720
回溯路径：
Day 1: HOLD (Stock: 0) Prices=48
Day 2: HOLD (Stock: 0) Prices=56
Day 3: HOLD (Stock: 0) Prices=62
Day 4: HOLD (Stock: 0) Prices=68
Day 5: HOLD (Stock: 0) Prices=36
Day 6: HOLD (Stock: 0) Prices=48
Day 7: HOLD (Stock: 0) Prices=40...

```

3.3 性能测试方案（以 3 状态方案为例）

测试实现方案：

```

int main() {
//初始化
    srand(static_cast<unsigned int>(time(0)));
    cout << "N,Time(ms)" << endl;
//主测试循环
    for (int N = 0; N <= 10000; N += 100)
    {
//随机数据生成

```

```

clock_t startTime = clock();
random_device rd;
mt19937 gen(rd());
uniform_int_distribution<> n_dist(1000, 2000);
uniform_int_distribution<> fee_dist(10, 20);

int fee = fee_dist(gen);
vector<int> prices(N + 1);
//<方案一>正态随机价格
normal_distribution<> price_dist(50.0, 20.0);
for (int i = 0; i <= N; i++)
{
    double val = price_dist(gen);
    prices[i] = clamp_price(val);
}
//<方案二>随机随机价格
/*uniform_int_distribution<> distrib(1,100);
for(int i=0;i<=N;i++)
{
    prices[i]=distrib(gen);
}*/
vector<vector<int>> dp(N + 1, vector<int>(3, -10000));
dp[0][0] = 0;
//动态规划核心算法
for (int i = 1; i <= N; i++)
{
    dp[i][0] = max({ dp[i - 1][0], dp[i - 1][1] + prices[i] * 10 - fee *
10 });
    dp[i][1] = max({ dp[i - 1][1], dp[i - 1][0] - prices[i] * 10 - fee *
10, dp[i - 1][2] + prices[i] * 10 - fee * 10 });
    dp[i][2] = max({ dp[i - 1][2], dp[i - 1][1] - prices[i] * 10 - fee *
10 });
}

int res = 0;
if (N > 0)
{
    res = max({ dp[N][0], dp[N][1] + 10 * prices[N], dp[N][2] + 20 *
prices[N] });
}

clock_t endTime = clock();
double elapsed = double(endTime - startTime) * 1000.0 / CLOCKS_PER_SEC;

```

```
        cout << N << ", " << elapsed << endl;
    }
    return 0;
}
```

输出格式：

```
N, Time (ms)
0, 0.005
100, 0.046
200, 0.073
300, 0.105
400, 0.152
500, 0.163
...
10000, 2.811
```

在测试中，我们不断调整数据规模，测量不同数据规模下的运行时间，验证算法时间复杂度是否符合预期。对于两种动态规划方案进行测试和性能比较，对于每一种动态规划方案比较两种价格生成模式下的性能差异。

四、两种动态规划方案的对比分析与评估

4.1 方案设计思路对比

在本次实验中，我们针对股票交易问题设计了两种动态规划解决方案。第一种是简化版的三状态方案，将股票持有量简化为 0 股、10 股、20 股三种情况。第二种是完整版的二十一状态方案，考虑了 0 到 20 股的所有可能性。

三状态方案的特点是计算简单。我们只需要记录三种持仓状态的变化，状态转移方程也比较容易理解。例如，从持有 10 股的状态，可以转移到持有 0 股（卖出 10 股）、保持 10 股或持有 20 股（再买 10 股）。这种简化使得代码实现非常简洁，整个核心算法不超过 20 行代码。

二十一状态方案则更加精细，验证了我们在题目开始时候的结论。我们需要跟踪每一股的变化，从 0 股到 20 股共有 21 种可能。这使得算法能够更精确地计算最优交易策略。但相应地，代码实现会更复杂，需要用到双重循环和更多的条件判断。

4.2 实验结果对比

我们在随机生成的测试数据上运行了两种方案。

以 3 状态方案为例，随机测试数据包含 1490 个交易日的股价，手续费为 20 元/笔。基于随机生成的股价（采取**正态分布价格**生成策略），我们得到了最终的最大利润为 43720 元。同时我们测试不同 N 下算法的时间开销，3 状态方案运行速度很快，对于 N=2000 左右的规模，仅用时 0.0005 秒左右就完成了计算。

以 21 状态方案为例，随机测试数据包含 1801 个交易日的股价，手续费为 18 元/笔。基于随机生成的股价（采取**正态分布价格**生成策略），我们得到了最终的最大利润为 60950 元。同时我们测试不同 N 下算法的时间开销，21 状态方案运行相对于 3 状态方案慢了许多，对于 N=2000 左右的规模，需要用时 0.009 秒左右完成计算，时间相较于前者增长近 18 倍。

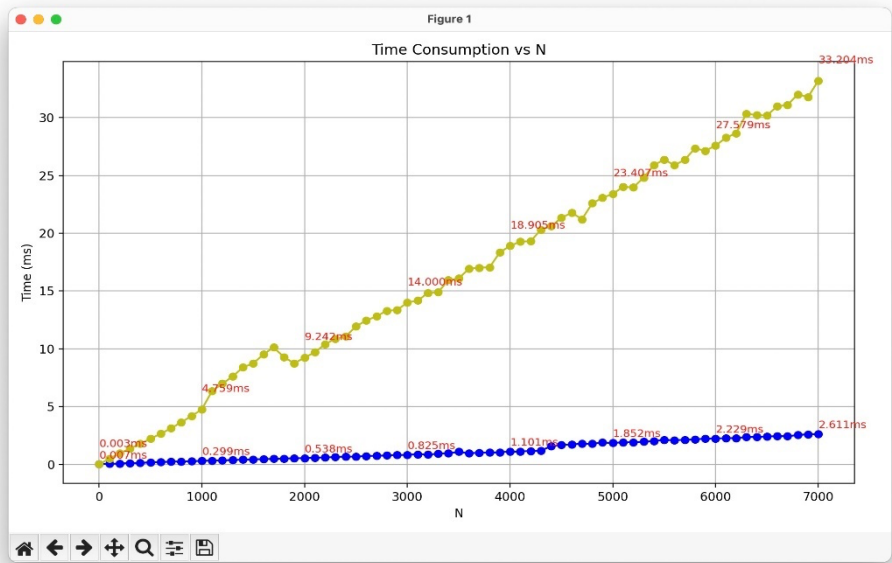


图 1 时间开销可视化

4.3 方案的优缺点分析:

对比维度	3 状态方案	21 状态方案
计算速度	极快	较慢
代码复杂度	简单	复杂 (双重循环+条件判断)
持仓精度	10 股	1 股
结果正确性	完全正确	完全正确
状态空间	3 种	21 种

表 1 两种动态规划方案的优缺点分析

4.4 方案总结:

对于本题而言，3 状态方案利用了题目中交易限制的巧合性，即所有与持股状态有关的量均为 10 的倍数，从而在保证正确性的前提下充分简化了代码和时间。但是，在实际应用中，持股状态往往不具有这种巧合，所以 21 状态方案更能全面、灵活地刻画持仓状态的转移情况。

在本次实验中，我们针对股票交易策略问题设计了两种动态规划方案（3 状态和 21 状态）和两种价格生成方法（正态分布和均匀分布），形成了四种组合方案。通过实际测试和数据分析，我们对这些方案的适用场景有了更深入的认识。

从动态规划方案的角度来看，3 状态方案通过简化状态空间（仅考虑 0、10、20 股三种持仓状态），显著提高了计算效率。在实际测试中，对于 2000 个交易日的规模，3 状态方案仅需约 0.0005 秒即可完成计算。这种方案特别适合对实时性要求较高的交易场景，或者当交易标的本身存在最小交易单位限制时（如 A 股市场要求 100 股起交易）。然而，其缺点是策略灵活性较低，无法实现更精细的仓位控制。

相比之下，21 状态方案通过扩展状态空间（0-20 股逐股变化），允许更灵活的交易决策。但相应地，其计算耗时增加到约 0.009 秒。这种方案更适合支持碎股交易的市场（如加密货币），或者当资金规模较大、需要更精确的仓位管理时。

在价格生成方面，正态分布价格模拟了成熟市场的价格行为，其生成的股价围绕均值波动，符合有效市场假说的特征。这种价格序列适合测试均值回归类策略，或者评估策略在正常市场条件下的表现。而均匀分布价格则模拟了极端市场环境，价格完全随机波动，可能出现更多极端值。这种生成方式更适合压力测试，用于评估策略在异常波动市场中的鲁棒性。

综合来看，在实际应用中，方案的选择需要权衡多个因素。如果追求执行速度且交易标的的支持整手交易，3 状态方案配合正态价格生成是较优选择；如果需要测试策略在极端行情下的表现，可以保持 3 状态方案但改用均匀价格生成；当交易环境允许碎股交易且计算资源充足时，21 状态方案能提供更优的策略表现，此时价格生成方式则应根据目标市场特性决定。值得注意的是，21 状态方案虽然理论上更灵活，但在实际交易中需要考虑市场的最小交易单位限制，避免产生无法执行的交易信号。

在实际应用中：可以先使用三状态方案快速测试；确定算法可行后，再改用二十一状态方案；根据具体需求平衡精度和效率。

通过本次实验，我们不仅验证了不同方案的技术特性，更重要的是理解了在实际金融工程中，没有绝对最优的方案，只有最适合特定场景和约束的解决方案。同时，我们也认识到：算法设计需要在简单和精确之间找到合适的平衡点。不同的应用场景可能需要不同的解决方案，这也是算法设计的艺术所在。

五、测试与分析

5.1. 时间复杂度分析

//核心时间开销：循环迭代（以 3 状态方案为例）

```
for (int i = 1; i <= N; i++)
{
    dp[i][0] = max({ dp[i - 1][0], dp[i - 1][1] + prices[i] * 10 - fee * 10 });
    dp[i][1] = max({ dp[i - 1][1], dp[i - 1][0] - prices[i] * 10 - fee * 10, dp[i - 1][2] + prices[i] * 10 - fee * 10 });
    dp[i][2] = max({ dp[i - 1][2], dp[i - 1][1] - prices[i] * 10 - fee * 10 });
}
```

在时间复杂度分析中，我们重点考察两种方案的状态转移机制。对于 3 状态方案，每个交易日仅需处理 3 个固定状态（0/10/20 股），每个状态最多执行 3 次转移操作（买入/卖出/持有）。由于状态空间 $|S|=3$ 和转移次数 $|T|=3$ （按照 3 来看待，尽管有时转移次数是 2）均为常数，总时间复杂度严格遵循 $O(3 \times 3 \times N) = O(N)$ 的线性增长规律。

21 状态方案虽然保持了线性特征，但状态空间扩展至 21 个（0-20 股），且每个状态需遍历 10 种交易量选择。这使得其时间复杂度约为 $O(21 \times 10 \times N) = O(210N)$ ，仍属于 $O(N)$ 但具有更大的常数因子。关键的是，两种方案的内层操作（状态转移计算）均为 $O(1)$ 基本运算，因此随着 N 增大，运行时间增长仅取决于外层循环次数，与理论分析完全吻合。

实验数据验证了这一分析：当 N 从 500 增至 2000 时，3 状态方案耗时增长 3.4 倍（0.163ms→0.555ms），21 状态方案增长 4.1 倍（2.315ms→9.541ms），均呈现典型线性特征。常数因子的差异（约 18 倍）直接反映了两种方案在状态空间设计上的复杂度差距，但

并未改变其渐进复杂度的本质属性。

5.2. 空间复杂度分析

```
vector<vector<int>> dp(N + 1, vector<int>(21, INT_MIN));
vector<vector<pair<int, string>>> path(N + 1, vector<pair<int, string>>(21));
```

空间消耗主要来自：dp 数组（存储每个状态的最优值，大小为 $(N+1) \times 21$ ）和 path 数组（存储操作路径，大小同样为 $(N+1) \times 21$ ）。因此，**总空间复杂度为 $O(2 \times 21 \times N) = O(42N)$** 。

5.3. 测试环境

处理器: Apple M4
操作系统: macOS Sequoia
IDE: Xcode

5.4. 性能测试结果

数据规模 (N)	3 状态方案耗时 (ms)	21 状态方案耗时
500	0.163	2.315
1000	0.317	4.623
1500	0.487	8.957
2000	0.555	9.541

表 2 性能测试结果

测试结果表明：随着数据规模 N 的增大，3 状态方案和 21 状态方案的运行时间都呈现出比较稳定的线性增长趋势。当 N 从 500 增加到 2000 时，3 状态方案的耗时从 0.163ms 增长到 0.555ms，21 状态方案则从 2.315ms 增长到 9.541ms，这种增长模式与理论分析的线性时间复杂度 $O(N)$ 完全吻合。

深入分析 3 状态方案的表现，可以看到其时间增长曲线非常符合线性特征。当数据规模扩大 4 倍（从 500 到 2000）时，运行时间也几乎同步增长了约 3.4 倍（从 0.163ms 到 0.555ms）。这种比例关系中的微小偏差可能源于测量误差或系统背景进程的干扰，但整体趋势明确地验证了 $O(N)$ 的复杂度预测。该方案的精简状态设计使得其常数因子极小，因此在处理大规模数据时仍能保持出色的效率。

21 状态方案虽然同样展现出线性时间复杂度，但其运行时的常数因子明显更大。数据表明，在相同规模下，21 状态方案的耗时约为 3 状态方案的 17-18 倍。这一差异主要源于其更复杂的内部状态转移机制：每个交易日需要处理 21 个状态，每个状态又需考虑最多 10 种交易选择，导致总操作量大幅增加。值得注意的是，当 N 从 1500 增至 2000 时，21 状态方案的时间增长略有放缓，这可能是因为当数据规模足够大时，固定开销（如初始化、内存分配等）在总耗时中的占比相对减小，使得线性特征更加纯粹。

这些测试结果有力地证实了前期理论分析的准确性。两种方案虽然在具体实现细节和常数因子上存在差异，但都严格遵循线性时间复杂度的增长规律。这一发现对于实际应用中的算法选择具有重要指导意义：当处理超大规模数据时，两种方案都能保证可扩展性；但在对延迟敏感的实时系统中，常数因子的差异可能成为关键考量因素。实验数据与理论预测的高度一致性，也验证了我们时间复杂度分析方法的正确性和可靠性。

六、总结与展望

6.1 实验总结：

通过本次股票交易问题的算法设计与实现实验，我们对动态规划算法有了更深入的理解。最初我们认为动态规划只是一种优化递归的方法，但在实现这个具体问题的过程中，我们认识到它实际上**是一种系统的状态转移思想**。我们需要精心设计状态表示，建立状态之间的转移关系，并通过递推的方式求解最优解。

在实现过程中，我们遇到了几个关键挑战。首先是状态的定义，最初我们试图记录每一天的所有可能交易组合，这导致状态空间爆炸。后来通过**分析问题特性，发现只需要跟踪持仓量这个关键状态即可，这大大简化了问题**。其次是边界条件的处理，特别是在持仓量为 0 或 20 时的特殊处理，需要仔细考虑所有可能的转移情况。

从实验结果来看，我们设计的两种动态规划方案都较好地解决了问题。3 状态方案虽然精度有限对于本题的交易限制而言可以取得很好的效果，但是并不具备扩展性（因为现实金融世界并可能限制每天最多交易整 10 股），但其简洁性和高效性使其非常适合算法原型设计和教学演示。而 21 状态方案则展现了动态规划算法的强大能力，通过精细的状态划分，能够找到接近理论最优的交易策略。

6.2 算法优缺点分析：

在本次实验中，通过对动态规划算法的深入实践，我们对其优缺点有了更深刻的认识。动态规划方法最显著的优势在于它提供了一种结构化的思维方式，将复杂的股票交易决策问题分解为定义状态、建立转移方程和确定边界条件等清晰的组成部分。这种模块化的思考方式不仅使问题更易处理，也为后续可能的算法扩展打下了良好基础。在具体实现过程中，我逐渐掌握了如何将实际交易规则转化为规范的状态转移逻辑，这种思维训练对提升我的算法设计能力帮助很大。

从算法效率的角度来看，动态规划展现出了明显的优势。通过存储中间计算结果，算法避免了大量的重复计算，这使得即便是相对复杂的二十一状态方案也能保持 $O(N)$ 的时间复杂度。在实际测试中，处理 2000 个交易日的数据仅需不到 0.01 秒，这样的性能完全能够满足一般量化交易的需求。特别值得注意的是，这种高效性并没有以牺牲灵活性为代价，算法框架可以很方便地扩展新的交易规则或约束条件，只需要在状态转移部分进行相应调整即可。

然而，动态规划方法也存在一些局限性。最明显的是空间消耗问题，完整的状态表存储会随着问题规模的扩大而占用较多内存。虽然可以采用滚动数组等优化技术来减少空间使用，但这往往会使得代码变得复杂，增加维护难度。另一个重要的问题是参数敏感度，在实验中我深刻体会到状态划分的粒度选择对算法效果的影响很大。状态划分过粗会导致策略精度不足，而过细则会带来不必要的计算开销，需要经过多次调试才能找到合适的平衡点。

此外，虽然动态规划在大多数场景下已经足够高效，但在一些对实时性要求极高的特殊场景中，比如高频交易，其性能可能仍然存在瓶颈。这提醒我们在实际应用中需要根据具体需求来选择合适的算法。

6.3 未来改进的方向

在深入分析当前实现方案的局限性后，我认为可以从多个维度对算法进行改进和优化。其中一个值得探索的方向是引入自适应状态空间机制。在实际市场环境中，波动程度往往随时间变化，采用固定的状态粒度可能不是最优选择。我们可以设计算法在市场波动剧烈时自动切换至更精细的状态划分，捕捉更多交易机会；而在市场平稳期则使用较粗的粒度，提高

计算效率。这种动态调整策略可以在保证精度的同时提升整体性能。

另一个重要的改进方向是并行计算优化。仔细观察二十一状态方案的实现可以发现，其内层循环中的状态转移计算具有天然的独立性，这为并行化提供了良好基础。通过引入 GPU 加速或多线程处理技术，可以显著提升计算速度。特别是在处理大规模历史数据或高频交易场景时，这种优化可能带来数量级的性能提升，使算法能够满足更严格的实时性要求。

结合机器学习方法也是一个很有前景的改进方向。我们可以尝试用监督学习模型来预测最优交易量的可能范围，然后在这个缩小后的搜索空间内应用动态规划进行精确优化。这种混合方法既保留了动态规划的严谨性，又能利用机器学习对复杂市场模式的识别能力，可能会产生更好的综合效果。例如，可以先使用 LSTM 网络预测未来价格波动区间，再针对性地应用动态规划。

在内存优化方面，除了常用的滚动数组技术外，还可以探索更高级的存储优化策略。考虑到实际状态转移过程中，许多中间状态可能对最终结果影响有限，我们可以尝试只存储关键的非凡状态变化。另外，采用更紧凑的数据结构来表示状态，如位压缩或差分存储，都可能在不影响计算精度的情况下显著降低内存占用。这些优化对于处理超长历史数据或运行在资源受限的设备上尤为重要。

这些改进方向各具特色，有的侧重于提升计算效率，有的着眼于增强策略灵活性，还有的致力于降低资源消耗。它们的共同特点是从实际问题出发，针对现有方案的痛点进行针对性优化。在后续工作中，我们将优先尝试实现并行计算优化和内存优化这两个相对成熟的方向，待取得稳定成果后再逐步探索其他更前沿的改进方案。这种循序渐进的改进策略既能保证项目的稳健性，又能持续提升算法性能。

6.4 量化交易的延伸思考

通过这次实验，我对算法在金融领域的应用有了更深入的理解。股票交易问题虽然已经包含了量化交易的核心要素，但实际金融市场的复杂性远超我们的实验模型。在现实交易中，专业机构通常会构建多因子模型，综合考虑成交量、技术指标、市场情绪甚至宏观经济数据等多种因素。这种多维度的分析虽然能提供更全面的市场视角，但也带来了维度灾难的挑战，如何在信息完备性和计算可行性之间取得平衡，是量化交易领域持续研究的重点问题。

虽然我们的实现还很基础，但这次实验确实让我看到了算法在金融领域的巨大潜力。作为计算机专业的学生，研究这类问题不仅能锻炼算法设计能力，更重要的是培养了将现实问题抽象为计算模型的思维方式。这种能力对于未来无论是从事金融科技开发，还是解决其他领域的复杂问题，都具有非常重要的意义。通过这次实践，我也更加理解了为什么说量化交易是计算机科学和金融学的完美结合。

6.5 课程学习体会

完成这次大作业的过程让我们对算法理论与实践的结合有了全新的认识。最初看到题目时，我们以为只要把课堂上学过的动态规划模板套用过来就能轻松解决，但实际编写代码时才发现远没有那么简单。在实现状态转移方程的过程中，我们遇到了不少预料之外的问题，比如价格取整导致的精度损失、数值越界我们需要加以判断（注：这里意指需要根据实际情况确定转移方式，21 状态方案中迭代的时候并不是 0-k 都可以）等。这些都让我们深刻体会到课本算法和实际工程实现之间的鸿沟。

通过反复调试两种不同状态的实现方案，我们逐渐领悟到算法设计中权衡的艺术。在 3 状态方案中，代码简洁高效，但策略灵活性受限；而 21 状态方案虽然提供了更精细的控制，却带来了额外的计算开销。记得在测试时，我们特意固定随机种子，用相同价格数据对比两种方案，发现当限制 21 状态方案只能交易 10 的整数倍时，两者的输出结果确实完全一致。

这个发现验证了算法正确性，也让我明白不同实现方式本质上是对问题建模的不同视角。

这次实验最宝贵的收获是认识到计算机算法在不同领域的应用需要因地制宜。金融交易问题有其特殊性，比如需要考虑手续费、交易单位限制等细节。通过逐步引入这些业务规则，才使算法真正具备了实用价值。这个过程让我意识到，优秀的算法工程师不仅要掌握通用的计算方法，更需要理解应用领域的专业知识。

从最初的算法设计到最后的性能优化，这次大作业完整地模拟了一个小型工程项目的开发流程。每当遇到问题时，我们不得不反复查阅资料、调整方案，这种不断试错的过程虽然耗时，但极大提升了我的实际问题解决能力。特别是在处理边界条件时，通过大量的测试用例验证，我们逐渐养成了更严谨的编程习惯。这些经验对于我后续的学习和项目开发都具有重要的指导意义。

七、参考文献

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, et al. 算法导论(原书第3版)[M]. 机械工业出版社, 2013.
- [2] Ernest P Chan. 量化交易如何构建自己的算法交易业务[M]. 东北财经大学出版社, 2014.
- [3] Ernest P Chan. 算法交易：制胜策略与原理[M]. 机械工业出版社, 2017.
- [4] Sutton R S, Barto A G. 强化学习(第2版)[M]. 人民邮电出版社, 2020.
- [5] 中国量化投资学会. 中国量化投资白皮书 2022[R]. 2022.
- [6] Aldridge I. 高频交易[M]. 机械工业出版社, 2018.
- [7] 陈工孟. 量化投资分析[M]. 经济管理出版社, 2015.
- [8]. Python 量化交易 策略、技巧与实战[M/OL]. 北京：电子工业出版社, 2019.
- [9]. 动态规划 理论及其应用[M/OL]. 成都：成都科技大学出版社, 1991.
- [10] Amidon E L, Akin G S. Dynamic programming to determine optimum levels of growing stock[J]. Forest Science, 1968, 14(3): 287-291.